Production Information System

CHAPTER 3: Data Modeling

Dr. Jomana Bashatah

Outline

1. Introduction
2. Entity-Relationship (E-R) Modeling
3. **Normalization**
4. **Case Study in Data Modeling**

3. Normalization

Introduction to Normalization

- When designing a data model, **attribute field redundancy** must be avoided as much as possible
- Prevents implementation problems with unnecessary data storage
- **Normalization** is the process of evaluating a model to control data redundancies
- Data redundancy causes problems with data integrity during insert, delete, and update operations
- These problems are called **anomalies**

3. Normalization

Types of Anomalies

Three types of anomalies:

1. **Insertion anomalies**
2. **Deletion anomalies**
3. **Update anomalies**

Let’s examine these using the PO_Vendor table example.

3. Normalization

3.1 Insertion Anomalies

Problem Description

When inserting new records into the PO_Vendor table:

- If the enterprise qualifies a new vendor and wishes to record vendor information in the database, the new record will start from V_Name
- The first five fields (PO-related) will be NULL values until a purchase order is assigned to the vendor
- **Cannot enter the vendor record** because PO_Number is the

3. Normalization

3.2 Deletion Anomalies

Problem Description

The deletion anomaly results in the **complete loss of information** when a related record is deleted from the database.

Example:

- If purchase order number 2596 is canceled and deleted from the database

- The vendor for this purchase order is Pasta Supply Inc.

3. Normalization

3.3 Update Anomalies

Problem Description

Consider the situation where a vendor changes its address:

- **Case 1** (normalized structure): One update to the set of attribute values required
- **Case 2** (PO_Vendor table): Multiple records need to be changed depending on the number of purchase orders assigned to the vendor

Problems:

- In large databases this becomes very unwieldy
- Risk of missing values that need to be updated
- Risk of introducing different values for the same data element due to data entry errors

3. Normalization

3.4 Normal Forms

- **Normalization** is the process of controlling data redundancy and eliminating anomalies
- Provides systematic rules to determine whether redundancy can be eliminated
- The process tries to uncover **functional dependencies** between attributes
- Provides rules to be applied for eliminating redundancies

3. Normalization

3.4 Normal Forms

Definition: A functional dependency between two attributes (A and B) exists if a value of A uniquely determines a value of B.

Identification: Can be identified from a data set by:
1. Identifying a value of A (“a”)
2. For each instance of that value, confirming it is associated with the same value of B (“b”)

Example: V_Name functionally determines V_Street - “Jersey Vegetable Co.” always corresponds to “2 Main St.”

3. Normalization

3.4 Normal Forms

Normal Form States

A data model exhibits a **normalization state**, called a **normal form**.

Three states discussed:

1. **First Normal Form (1NF)**

2. **Second Normal Form (2NF)**

3. **Third Normal Form (3NF)**

Goal: Revise entities from 1NF or 2NF to 3NF

3. Normalization

3.4 Normal Forms

Dependencies to Eliminate

In moving from 1NF to 3NF, two types of dependencies must be eliminated:

1. **Partial dependency**: Dependency based on part of a composite primary key
2. **Transitive dependency**: Dependency among nonkey attributes

3. Normalization

3.4 Normal Forms - First Normal Form (1NF)

First normal form has the following characteristics:

1. The **key attributes are defined**
2. **All attributes are dependent** on the primary key(s)
3. **No composite attributes exist** (each row/column intersection contains only one value, rather than a set of values)

3. Normalization

3.4 Normal Forms - First Normal Form (1NF)

Consider the relationship:

```
PO_Report (Order_number, Order_date,
Customer_number,
          Customer_country, Product_number,
Product_price,
          Product_Quantity)
```

Primary Key: (Order_number, Product_number)

This table is in 1NF but can contain **partial dependencies**, leading to data redundancy.

3. Normalization

3.4 Normal Forms - Second Normal Form (2NF)

A table is in **second normal form (2NF)** if:

1. It is in **1NF**

2. There are **no partial dependencies** based on the primary key(s)

3. Normalization

3.4 Normal Forms - Second Normal Form (2NF)

```
PO_Report (Order_number, Product_number,
Order_date,
          Customer_number, Customer_country,
Product_price,
          Product_Quantity)
```

Partial dependencies found: - Product_Number → Product_price - Order_number → Order_date - Order_number → Customer_number - Order_number → Customer_country

Conclusion: The table is **NOT in 2NF**

3. Normalization

3.4 Normal Forms - Second Normal Form (2NF)

A table is converted to second normal form by eliminating partial dependencies

There are two steps in the process as follows:

Process:

1. Write each key component that has partial dependency on a separate line and the original key on the last line

2. Define an entity name for each primary key and write the primary key attribute and dependent attributes

Result:

```
PO_Report1 (Order_number, Order_date, Customer_number,
Customer_country)
PO_Report2 (Product_number, Product_price)
PO_Report3 (Order_number, Product_number, Product_Quantity)
```

3. Normalization

3.4 Normal Forms - Second Normal Form (2NF)

Although there are no partial dependencies based on primary key attributes, there are **dependencies within non-key attributes**.

Example: In PO_Report1

`Customer_number → Customer_Country`

This dependency between non-key attributes is known as a **transitive dependency**.

Result: PO_Report1 is **NOT in 3NF**

3. Normalization

3.4 Normal Forms - Third Normal Form (3NF)

A table is in **3NF** if:

1. It is in **2NF**

2. It contains **no transitive dependencies**

When transitive dependencies are eliminated, a table has been converted to third normal form.

3. Normalization

3.4 Normal Forms - Third Normal Form (3NF)

A table is converted to third normal form by eliminating transitive dependencies

There are two steps in the process as follows:

Process:

1. Keep in the entity the attributes that **depend directly on the key**
2. Define in another table the attributes that are **transitively dependent** on the key. The transition attribute remains duplicated in the initial entity and becomes the primary key of the new entity

3. Normalization

3.4 Normal Forms - Third Normal Form (3NF)

From:

```
PO_Report1 (Order_number, Order_date,
Customer_number, Customer_country)
```

Step 1: Keep directly dependent attributes

```
PO_Report1-1 (Order_number, Order_date,
Customer_number)
```

Step 2: Create new entity for transitive dependency

```
PO_Report1-2 (Customer_number,
Customer_country)
```

3. Normalization

3.4 Normal Forms - Third Normal Form (3NF)

The conversion process results in the following table definitions:

```
Order (Order_number, Order_date,
Customer_number)

Customer (Customer_number, Customer_country)

Product (Product_number, Product_price)

Product_Order (Order_number, Product_number,
Product_Quantity)
```

All tables are now in **Third Normal Form (3NF)**.

4. Case Study in Data Modeling

Scenario

We illustrate the process of developing an E-R model using a hypothetical organization responsible for managing materials inventoried by the firm.

Information from interviews with key individuals:

4. Case Study in Data Modeling

Requirement 1: Material Types

The department is responsible for inventory of three material types: raw material, finished goods, and supplies.

- Each material, identified by a **material ID**, belongs to **one material type**
- Each material type classifies **many different materials**

Entities identified: - **MATERIAL_TYPE** - defines a class of materials - **MATERIAL** - with primary key of material ID

4. Case Study in Data Modeling

Requirement 2: Purchase Orders

When a raw material or supply is ordered, it appears on purchase order detail (PO_DETAIL). Purchase orders are also used for capital equipment and services.

Relationships:

- MATERIAL is related to PO_DETAIL (**1:M relationship**)
- MATERIAL is **optional** (finished goods not ordered on PO)
- PO_DETAIL is **optional** (some details are for capital goods/services)

4. Case Study in Data Modeling

Requirement 3: Material Lots

When materials arrive or finished goods are produced, they become **material lots** with unique material lot numbers.

Relationships:

- For each material: **zero, one, or many** material lots
- Each material lot: manifestation of **one and only one** material
- MATERIAL has **optional** relationship to MATERIAL_LOT
- Each material lot **must participate** in relationship to material
- MATERIAL_LOT relates to PO_DETAIL (optional, as some lots are finished goods)

4. Case Study in Data Modeling

Requirement 4: Superclass-Subclass

MATERIAL_LOT includes:

- Inventoried raw materials
- Supplies
- Finished goods

These three categories are **nonoverlapping** and each material lot must appear in **one and only one** category.

Rationale for subclasses:

- Common attributes: material ID, description, quantity on hand
- Different attributes: purchased materials relate to vendors/POs; finished goods relate to production lines

4. Case Study in Data Modeling

Requirement 5: Warehouse Storage

A material lot is stored in a warehouse location.

Relationships:

- A warehouse location may store **zero, one, or many** material lots
- A material lot may occupy **more than one location** if too large
- A material lot may **not be assigned** to a warehouse location at some points
- **Cardinality**: M:N relationship
- Both entities are **optional** in the relationship

4. Case Study in Data Modeling

Requirement 6: Production Lines

If a material lot is finished goods, it was produced on a production line.

Relationships:

- A production line can produce **more than one material lot**
- Each lot of finished goods produced on **one and only one** production line
- Relationship is between subclass **FINISHED_GOOD** and **PRODUCTION_LINE**
- **Cardinality**: 1:M relationship

4. Case Study in Data Modeling

Requirement 7: Equipment

A production line is made up of one or more pieces of equipment.

Relationships:

- Each piece of equipment has its own **equipment ID**
- Each piece of equipment belongs to **one and only one** production line
- **Cardinality**: 1:M relationship
- **Mandatory** on both sides

4. Case Study in Data Modeling

Requirement 8: Quality Control Tests
Materials may have quality control tests associated with them.
Details:

- True of **some raw materials** and **all finished goods**

- Some raw materials and all supplies are **not subject** to any tests
- Some tests performed on **more than one material**
- Some materials require **more than one test**
- **Cardinality**: M:N relationship
- MATERIAL is **optional**; QUALITY_CONTROL_TEST is **mandatory**

Conclusion

- We introduced the **entity-relationship approach** to developing a data model of the enterprise
- The E-R modeling methodology is the **basis for other formats** used for designing data models in industry
- In industry, **computer-aided software engineering (CASE) tools** are widely used to document information system design projects and help designers be more productive

Summary

Key concepts covered:

1. **ANSI/SPARC three-level architecture**
2. **Entity-Relationship modeling primitives**
3. **Normalization process** (1NF → 2NF → 3NF)
4. **Practical case study** in data modeling